

GEN Z STUDIO

Mobile Application Platform

CHAPTER 5

Mobile Application Development & Architecture

Flutter · AWS Amplify · Amazon Cognito
AWS AppSync · DynamoDB · Amazon S3 · EC2

Graduation Project — Module 2: Mobile Application

Table of Contents

5.1 Introduction

5.2 Technology Stack & Development Framework

- 5.2.1 Flutter -- In-Depth Framework Analysis
- 5.2.2 AWS Amplify -- Cloud Backend Integration Layer
- 5.2.3 Key Dependencies

5.3 Application Architecture

- 5.3.1 High-Level Architecture Overview
- 5.3.2 Application Startup & Navigation Flow
- 5.3.3 Code Organization & Module Structure
- 5.3.4 State Management Strategy

5.4 Authentication & User Management

5.5 Client-Facing Features & User Interface

- 5.5.1 Responsive Layout System
- 5.5.2 Splash / Welcome Screen UI
- 5.5.3 Home Screen -- Welcome Banner
- 5.5.4 Studio Grid -- Responsive Display
- 5.5.5 Services Catalog -- Category-Grouped Design
- 5.5.6 Booking Form -- Detailed UI Walkthrough

5.6 AI-Powered Chatbot -- Screen Design & Technical Implementation

- 5.6.1 Chatbot Screen UI
- 5.6.2 Empty State -- Welcome Screen
- 5.6.3 Message Bubbles
- 5.6.4 Loading Indicator
- 5.6.5 Message Input Area
- 5.6.6 Backend Communication & Session Management
- 5.6.7 Booking Intent Detection & Execution

5.7 Real-Time Support Chat & Ticket System

5.8 Employee Administration Dashboard

- 5.8.1 Dashboard Tab -- Booking Management
- 5.8.2 Notification System -- Employee Side
- 5.8.3 Wide-Screen Layout

5.9 Notifications System

5.10 Analytics & Reporting

5.11 Internationalization & Responsive Design

5.12 Database Schema & Data Models

5.13 Real-Time Synchronization Strategy

5.14 User Interface Design -- Visual Language & Design System

5.15 Summary

CHAPTER 5: GEN Z STUDIO MOBILE APPLICATION

5.1 Introduction

The Gen Z Studio Mobile Application is the primary commercial and operational interface of the entire enterprise platform. While the network infrastructure (Chapters 3 & 4) provides the secure physical backbone, and the cloud module provides scalable backend services, the mobile application is the component every stakeholder — client or employee — interacts with directly, every day. It is therefore the most visible and commercially critical piece of the system.

The application was developed using **Flutter**, Google's open-source cross-platform UI toolkit, and its backend runs entirely on **Amazon Web Services (AWS)** through the AWS Amplify framework. This combination yields an application that is simultaneously cross-platform, cloud-native, and real-time — all qualities essential for a professional studio booking platform operating in a competitive market.

User Roles

The application serves two distinct user categories, each with a completely different set of screens, features, and data access rights:

Clients (Customers) — End users who browse studio listings, book sessions, track booking history, consult the AI chatbot assistant, and communicate with staff through a real-time support channel.

Employees (Administrators) — Studio staff who manage incoming bookings (approve/reject/mark as done), respond to client support requests, maintain studio and service listings, view financial analytics, and access business reports.

At application startup, the system reads the user's Cognito group from the JWT token and silently routes them to the appropriate interface — no manual role selection required.

The following sections describe the complete technical architecture, the detailed rationale behind every major technology choice, a full walkthrough of every screen and feature, and the data synchronization strategy that powers the application's real-time behavior.

5.2 Technology Stack & Development Framework

5.2.1 Flutter — In-Depth Framework Analysis

Flutter is Google's open-source UI toolkit for building natively compiled applications for mobile, web, and desktop from a **single codebase**. First released publicly in 2018, it has become one of the most widely adopted cross-platform frameworks in the industry, used in production by Google Pay, BMW, eBay, Nubank, and Alibaba at scale.

What Flutter Is — Under the Hood

Flutter is composed of two main layers:

1. The Dart SDK

Flutter applications are written entirely in Dart — a modern, strongly-typed, ahead-of-time compiled language developed by Google. Dart was chosen by the Flutter team for several specific reasons:

- It compiles to native ARM machine code for Android and iOS (no interpreter at runtime).
- It compiles to optimized JavaScript for Web targets via `dart2js`.
- Its `async/await` and `Stream` primitives are first-class language features, not library add-ons, which makes writing reactive, event-driven UI code significantly cleaner than JavaScript callback patterns.
- Its strong static type system catches errors at compile time, before the app runs.

2. The Flutter Rendering Engine

Flutter does not use the host platform's native UI components. It does not use Android's `View` system or iOS's `UIKit`. Instead, Flutter embeds its own rendering engine (built on Skia, or Impeller on newer versions) that draws every pixel on the screen itself, directly to the GPU canvas provided by the operating system.

This design choice has a profound consequence: **every Flutter application looks and behaves identically on Android, iOS, and Web**, because the rendering is entirely self-contained. There are no platform-specific rendering differences, no font rendering inconsistencies between Android and iOS, and no component behavior that varies by OS version.

The Widget Tree Model

In Flutter, every visual element — from a single text label to a full-screen layout — is a **widget**. Widgets are immutable descriptions of how to render part of the UI. They are organized in a tree (the widget tree), and Flutter rebuilds subtrees of this tree efficiently when state changes.

There are two fundamental widget types:

Widget Type	Description	Example in Gen Z Studio
<code>StatelessWidget</code>	Appearance is determined entirely by its constructor arguments. Never changes after being built.	Studio price label, section header text
<code>StatefulWidget</code>	Paired with a <code>State</code> object. When <code>setState()</code> is called, Flutter marks the widget as dirty and rebuilds it.	Client home screen (reacts to new studio data), booking form (updates price in real time)

When `setState()` is called on a `StatefulWidget`, Flutter's reconciler compares the new widget tree to the previous one (the "diffing" process), and only updates the parts of the rendered scene that actually changed. This makes UI updates fast even on low-end devices.

Why Flutter Was Chosen for Gen Z Studio — Six Specific Reasons

The selection of Flutter over alternatives (React Native, Xamarin, Ionic, or separate native applications) was deliberate and driven by requirements specific to this project:

Reason 1 — Official AWS Amplify Flutter SDK

Amazon maintains and publishes an official Flutter SDK for the entire Amplify ecosystem: `amplify_flutter`, `amplify_auth_cognito`, `amplify_api`, and `amplify_storage_s3`. These packages provide idiomatic Dart APIs for Cognito authentication, AppSync GraphQL operations, and S3 file storage. Using the official SDK meant that the Gen Z Studio backend integration was built on tested, maintained infrastructure rather than custom HTTP wrappers. No other cross-platform framework has this level of first-party AWS support.

Reason 2 — True Single Codebase for Three Platforms

The studio serves clients on Android phones, iOS phones, and desktop web browsers. Flutter produces three deployment targets from one Dart codebase: an Android `.apk/.aab`, an iOS `.ipa`, and a web bundle. The alternative — three separate codebases — would require three separate development, testing, and maintenance efforts, which is prohibitive for a graduation-level project team.

Reason 3 — Dart Streams and the Real-Time Architecture

The Gen Z Studio application has significant real-time requirements: employees need to see new bookings the moment they arrive, clients need notification banners to appear without refreshing, and studio availability must stay current. Dart's `Stream` and `StreamSubscription` system integrates naturally with AppSync WebSocket subscriptions. The following pattern, used throughout the application, is idiomatic Dart:

```
StreamSubscription? _bookingsSubscription;
```

```
_bookingsSubscription = Amplify.API.subscribe(

  request: ModelSubscriptions.onCreate(BookingRequest.classType),

  onData: (event) {

    final booking = event.data;

    if (!mounted) return;

    setState(() => bookingRequests.insert(0, booking.toMap()));

    _showNewBookingBanner(booking.toMap());

  },

);
```

This reactive pattern would require significantly more boilerplate in JavaScript/React Native.

Reason 4 — AOT Compilation and Performance

In release builds, Flutter's Dart code is compiled ahead-of-time to native ARM machine code. This eliminates the JavaScript-to-native bridge overhead that is the primary performance bottleneck in React Native. The result is scroll animations that maintain 60 fps, form input that responds instantly, and image loading that does not cause dropped frames — all without manual performance optimization.

Reason 5 — Hot Reload for Rapid UI Development

During development, any change to the Dart source code — whether it is a color value, a layout padding, or a business logic condition — is reflected in the running application in under one second, **without losing the current application state**. This means a developer can be on the booking form, change a border radius or adjust a label, and see the result immediately while remaining on that screen. This acceleration is not possible with native development (which requires full recompilation and relaunch) or with some web-based frameworks.

Reason 6 — Material Design System Built-In

Flutter ships with a complete, production-quality implementation of Google's Material Design system. Every component in the Gen Z Studio application — cards, dialogs, bottom sheets, snackbars, floating action buttons, date pickers, navigation drawers, `LinearProgressIndicator` loading bars — is a standard, accessibility-tested Flutter widget. The project's custom navy/blue color palette was applied globally through `ThemeData`, ensuring zero visual inconsistency between components.

Flutter Compilation Targets

Target Platform	Compilation Method	Binary Output
Android	AOT — Dart to ARM native	.apk / .aab bundle
iOS	AOT — Dart to ARM64 native	.ipa archive
Web	Dart transpiled to JavaScript via <code>dart2js</code>	HTML + JS + assets bundle
Windows / macOS / Linux	AOT — Dart to x86-64 native	Native executable

For this project, **Android** and **iOS** are the primary mobile targets. The **Web** target serves as a secondary administrative interface accessible from desktop browsers without installing an application.

Flutter vs. Alternatives — Comparison

Criterion	Flutter	React Native	Separate Native Apps
Single codebase	Yes — Android, iOS, Web, Desktop	Yes — Android, iOS	No — one per platform
Rendering	Own GPU renderer (pixel-perfect)	Native components (platform-dependent)	Full native
Performance	Near-native (AOT, no JS bridge)	Near-native (JS bridge overhead)	Full native
Official AWS Amplify SDK	Yes — <code>amplify_flutter</code>	Yes — <code>aws-amplify</code>	Per-platform (Kotlin/Swift)
Real-time (WebSocket/Stream)	First-class Dart Streams	Supported (JS EventEmitter)	Native callbacks
Development speed	Fast (Hot Reload + single codebase)	Fast (Hot Reload + single codebase)	Slow (2–3× the work)
Type safety	Fully typed (Dart)	Optional (TypeScript)	Fully typed (Kotlin/Swift)
UI consistency across platforms	Pixel-perfect identical	Minor platform differences	Platform-specific look

5.2.2 AWS Amplify — Cloud Backend Integration Layer

AWS Amplify is a set of tools and services that connect the Flutter frontend to the full power of AWS without requiring the developer to manually configure IAM policies, write API Gateway routes, or manage Cognito flows. Amplify handles the following for Gen Z Studio:

Authentication: The `amplify_auth_cognito` plugin manages the complete Cognito authentication lifecycle — sign-up, email verification, sign-in, JWT token storage, silent token refresh, and sign-out — through a clean Dart API (`Amplify.Auth.signIn()`, `Amplify.Auth.signOut()`, `Amplify.Auth.fetchAuthSession()`).

GraphQL API: The `amplify_api` plugin connects to the AWS AppSync endpoint. The Flutter app calls `Amplify.API.query()`, `Amplify.API.mutate()`, and `Amplify.API.subscribe()` without writing HTTP request boilerplate. AppSync handles authorization, caching, and DynamoDB resolver execution on the backend.

File Storage: The `amplify_storage_s3` plugin provides upload, download, and URL-generation APIs for Amazon S3. Studio images and user profile photos are managed through `Amplify.Storage.uploadData()` and `Amplify.Storage.getUrl()`.

Code Generation: The Amplify CLI reads the GraphQL schema definition file (`amplify/backend/api/genz/schema.graphql`) and auto-generates strongly-typed Dart model classes in `lib/models/`. This means `BookingRequest`, `Studio`, `AppNotification`, `ChatMessage`, and other data types are available as typed Dart objects — no manual JSON parsing or serialization.

Amplify Initialization at App Startup

Amplify is configured once in `main.dart` before the Flutter widget tree is created. `DataStore` was deliberately disabled because the owner-based `@auth` rules in the GraphQL schema prevent `DataStore` from performing its bulk sync of records that belong to other users. The application uses direct `Amplify.API` calls instead:

```
final List<AmplifyPluginInterface> plugins = [

  AmplifyAPI(

    options: APIPluginOptions(modelProvider: ModelProvider.instance),

  ),

  AmplifyAuthCognito(),

  AmplifyStorageS3(),

];
```



```
await Amplify.addPlugins(plugins);

await Amplify.configure(amplifyconfig);
```

5.2.3 Key Dependencies

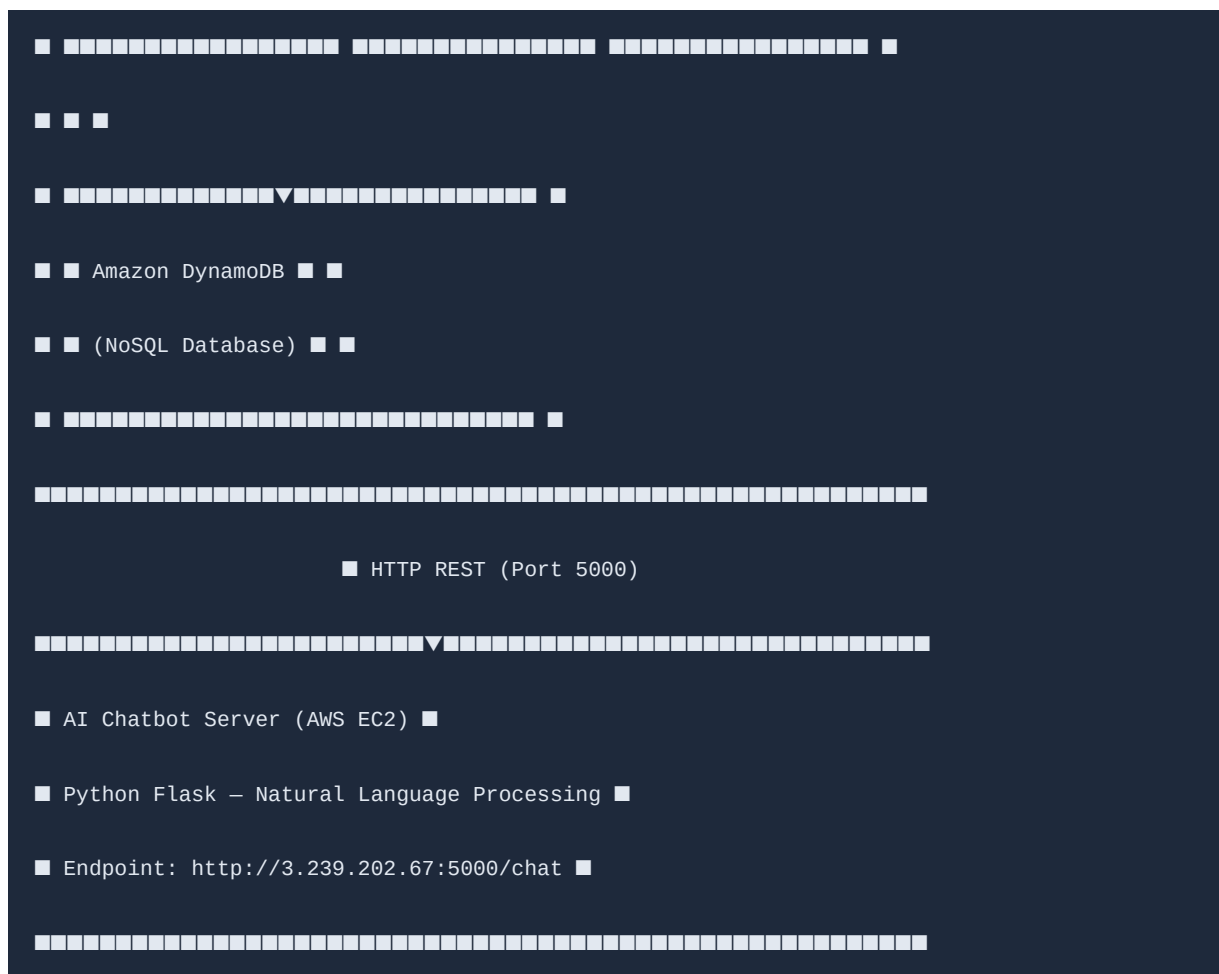
Package	Version	Purpose
amplify_flutter	^2.x	Core AWS Amplify integration and initialization
amplify_auth_cognito	^2.x	Cognito user authentication (sign-in, tokens, session)
amplify_api	^2.x	GraphQL query, mutation, and subscription operations
amplify_storage_s3	^2.x	S3 file upload and download for images
provider	^6.x	Global state management (theme, locale, user state)
shared_preferences	^2.x	Local device storage for session, chatbot history, settings
http	^1.x	HTTP client for AI chatbot EC2 server communication
intl	^0.19.x	Date/number formatting and internationalization support
pdf	^3.x	PDF report generation for employee analytics exports
image_picker	^1.x	Camera/gallery image selection for profiles and studios
fl_chart	^0.67.x	Bar charts, line charts, and pie charts for analytics
flutter_localizations	SDK	RTL support, Material localization delegates

5.3 Application Architecture

5.3.1 High-Level Architecture Overview

The Gen Z Studio application follows a **Client-Server architecture** with a fully cloud-native backend. The complete system spans three tiers:





All data between Flutter and AWS services is encrypted with TLS (HTTPS). Real-time event streams use WebSocket connections through GraphQL Subscriptions — AppSync automatically upgrades an HTTPS connection to a WebSocket when a subscription is opened. The EC2 chatbot server communicates via standard HTTP REST.

5.3.2 Application Startup & Navigation Flow

The application startup sequence is defined in `main.dart` and follows a structured bootstrap process:

App Launch (`main.dart`)

↓

`WidgetsFlutterBinding.ensureInitialized()`

↓

SystemChrome – transparent status bar, portrait lock (mobile only)

↓

SettingsService.loadSettings() – restore saved theme & locale

↓

_configureAmplify() – register Auth, API, Storage plugins

↓

runApp() with MultiProvider (AppState)

↓

SplashScreen displayed (navy background, camera icon, "GENZ Studios" text, spinner)

↓

Amplify.Auth.fetchAuthSession()

■■■ Not signed in → WelcomeScreen (login/register)

■■■ Signed in

↓

AWSStorageService.loadCurrentUser() – fetch name, email, type, image

↓

AppState.setUser() – populate global state

↓

User type check (from Cognito JWT group)

■■■ "Employees" → EmployeesScreen

■■■ "Clients"

↓

SharedPreferences check: onboarding_done?

```
■■■ No → OnboardingScreen → ClientScreen
```

```
■■■ Yes → ClientScreen directly
```

The **SplashScreen** is the application's initial loading indicator. It displays a full-screen navy background with a large white camera icon, the "GENZ Studios" title in bold white 28pt text, and a circular **CircularProgressIndicator** (white, 3px stroke weight) centered below it. This splash state persists only for the ~300ms required to check the auth session and load the user profile.

The **OnboardingScreen** is shown only on first launch after registration — it walks new clients through the application's key features before landing them on the home screen.

5.3.3 Code Organization & Module Structure

```
lib/

■■■ main.dart – Entry point, Amplify init, SplashScreen, routing

■■■ amplifyconfiguration.dart – Auto-generated Amplify config (endpoints, region)

■

■■■ models/

■ ■■■ ModelProvider.dart – Auto-generated GraphQL model registry

■ ■■■ BookingRequest.dart – Booking model (auto-generated)

■ ■■■ Studio.dart – Studio model (auto-generated)

■ ■■■ AppNotification.dart – Notification model (auto-generated)

■ ■■■ ChatMessage.dart – Support chat message model (auto-generated)

■ ■■■ GenzService.dart – Service catalog model

■

■■■ data/

■ ■■■ aws_storage.dart – AWSStorageService: all Amplify API calls

■
```

```
■■■ providers/

■ ■■■ app_state.dart – AppState: ChangeNotifier for global user state

■

■■■ services/

■ ■■■ app_localizations.dart – EN/AR translation string map + delegate

■ ■■■ settings_service.dart – ValueNotifier for theme & locale (no rebuild cascade)

■ ■■■ chatbot_booking_service.dart – AI intent parser + booking executor

■

■■■ theme/

■ ■■■ app_theme.dart – AppTheme.light(), AppTheme.dark(), AppColors constants

■

■■■ screens/

    ■■■ frist_screen.dart – WelcomeScreen (login + registration tabs)

    ■■■ onboarding_screen.dart – First-launch feature tour

    ■■■ client_screen.dart – Client: home, studio grid, booking form

    ■■■ Employees_screen.dart – Employee: booking management, support, studios

    ■■■ StudioDetailScreen.dart – Studio image gallery and details

    ■■■ ChatbotScreen.dart – AI chatbot conversation interface

    ■■■ chat_screen.dart – Human support chat thread

    ■■■ NotificationsScreen.dart – In-app notifications list

    ■■■ BookingDetailScreen.dart – Employee: detailed booking view

    ■■■ ReportsScreen.dart – Employee: analytics and PDF export

    ■■■ profile_screen.dart – User profile and settings
```

```
settings_screen.dart – App-wide settings
```

The `AWSStorageService` class in `data/aws_storage.dart` is the single facade for all backend communication. Every screen calls methods like `AWSStorageService.loadStudios()`, `AWSStorageService.saveBookingAtomic()`, or `AWSStorageService.sendNotification()`. No screen constructs GraphQL strings directly or calls `Amplify.API` from within widget code. This separation means the entire backend interaction contract is defined in one place, independently testable and replaceable.

5.3.4 State Management Strategy

Level	Mechanism	Used For
Global (cross-screen)	<code>Provider</code> + <code>AppState</code>	Current user data (email, name, type, image)
Global (reactive, lightweight)	<code>ValueListenable</code> + <code>ValueNotifier</code>	Theme mode, locale — triggers only widget rebuilds that <code>ValueListenableBuilder</code>
Screen-local	<code>StatefulWidget</code> + <code>setState()</code>	Tab index, form data, loading states, local list data
Real-time event stream	<code>StreamSubscription</code> (Dart async)	GraphQL WebSocket subscriptions for live booking/notification events
Periodic refresh	<code>Timer.periodic</code>	Polling-based fallback for clients (studios every 90s, bookings every 12s)

The `SettingsService` uses `ValueNotifier<ThemeMode>` and `ValueNotifier<Locale>` rather than `Provider` for theme and locale, because these values can be listened to directly by `ValueListenableBuilder` in `MyApp`'s `build()` method — this causes only the top-level `MaterialApp` to rebuild when the theme changes, not every widget that calls `context.watch<AppState>()`. This prevents an unnecessary full widget tree rebuild on a theme toggle.

Text scaling is globally clamped at app startup to between 85% and 120% of the system font scale:

```
data: mq.copyWith(
  textScaler: TextScaler.linear(
```

```
mq.textScaler.scale(1).clamp(0.85, 1.2),  
  
),  
  
),
```

This prevents the UI from breaking when a user has set an extreme system accessibility font size.

5.4 Authentication & User Management

Authentication is one of the most critical subsystems in any commercial application. The Gen Z Studio platform handles identity management entirely through **Amazon Cognito**, integrated into the Flutter application via the `amplify_auth_cognito` plugin. This approach was chosen deliberately to avoid the security and maintenance burden of building a custom authentication server — Cognito is a fully managed, battle-tested identity provider operated by AWS and compliant with industry security standards including OWASP best practices.

5.4.1 Amazon Cognito — Architecture Overview

Cognito is Amazon's identity-as-a-service platform. In the Gen Z Studio architecture, a single **Cognito User Pool** stores all registered users (both clients and employees). A Cognito User Pool is a directory that holds user identities, enforces password policies, manages email verification, handles JWT token lifecycle, and provides OAuth 2.0-compatible token issuance.

When a user registers or signs in, Cognito issues three tokens:

Token	Lifetime	Contents	Used For
ID Token	1 hour	User identity claims: <code>email</code> , <code>name</code> , <code>cognito:groups</code> , custom attributes	Identity verification in Flutter; passed to AppSync for <code>@auth</code> enforcement
Access Token	1 hour	Scopes, user sub, token type	AWS service authorization (API Gateway, AppSync direct access)
Refresh Token	30 days	Opaque refresh credential	Silent token renewal — Amplify calls this automatically when ID/Access tokens expire

The Amplify SDK stores all three tokens in platform-specific secure storage: **Android Keystore** on Android devices and **iOS Keychain** on iPhones. Neither token is stored in plain `SharedPreferences` or exposed to application code as a raw string — Amplify's `AmplifyAuthCognito` plugin handles all storage and retrieval internally. This means even if the device's file system were compromised, the tokens cannot be read by a third-party process.

5.4.2 Registration and Email Verification Flow

The registration flow is implemented in `frist_screen.dart` (the `WelcomeScreen`) and follows Cognito's standard confirm-then-sign-in pattern:

```
User fills: first name, last name, email, password, confirm password
↓
Amplify.Auth.signUp(username: email, password: password, options: ...)
↓
Cognito sends a 6-digit OTP to the user's email address
↓
App shows OTP confirmation screen
↓
Amplify.Auth.confirmSignUp(username: email, confirmationCode: otp)
↓
Cognito marks the account as CONFIRMED
↓
App automatically signs in: Amplify.Auth.signIn(username: email, password: password)
↓
JWT tokens issued → routing to ClientScreen (new users default to Client group)
```

The email verification step ensures that only real email addresses can hold accounts — an unverified account cannot sign in. This prevents spam registrations and ensures that booking confirmation

notifications and support messages reach a valid email.

Custom user attributes (`given_name`, `family_name`) are passed in the `signUp` options so that Cognito stores the user's display name alongside the credential. These are retrieved later via `Amplify.Auth.fetchUserAttributes()` to populate the profile screen and the welcome banner.

5.4.3 Sign-In and Silent Token Refresh

Sign-in calls `Amplify.Auth.signIn(username: email, password: password)`. On success, Amplify caches the session internally. On subsequent app launches, `Amplify.Auth.fetchAuthSession()` checks whether a valid session already exists — if the ID Token has not expired, the user is signed in silently without re-entering credentials. If the ID Token has expired but the Refresh Token is still valid (within 30 days), Amplify automatically exchanges the Refresh Token for new ID and Access Tokens. The user never sees a "please log in again" screen unless they have not opened the application for 30 days.

5.4.4 Role-Based Routing via Cognito Groups

Two user groups are defined in the Cognito User Pool:

Cognito Group	Permissions
<code>Clients</code>	Read studios and services; create and read own bookings; create and read own notifications; send support messages
<code>Employees</code>	Full CRUD on studios, services, and bookings; read all notifications (including the employee inbox); read and write all support messages

After sign-in, the application reads the `cognito:groups` claim from the decoded JWT ID Token to determine the user's role. This is done through `AWSStorageService.loadCurrentUser()`, which calls `Amplify.Auth.fetchUserAttributes()` and resolves the user's type to either `'employee'` or `'client'`. The `AppState.isEmployee` boolean gates the navigation routing in `SplashScreen._bootstrap()`.

AppSync Authorization Enforcement

The `@auth` rules defined in the GraphQL schema enforce access control at the API layer, not just in the Flutter client. Even if a malicious client bypassed the Flutter routing, AppSync would reject unauthorized mutations:

- A client attempting to query all bookings (not just their own) receives an empty result

- A client attempting to mutate a studio record receives an authorization error
- A client attempting to read another client's notifications is blocked at the resolver level

This means the application's security does not depend on the correctness of the Flutter UI — the backend enforces it independently.

5.5 Client-Facing Features & User Interface

The client interface (`client_screen.dart`) is the primary commercial surface of the application. It is structured around two main views, navigated via a tab bar or sidebar: the **Home View** (studio discovery + services catalog) and the **Booking View** (reservation form).

5.5.1 Responsive Layout System

The client screen adapts to three screen size tiers using a single conditional expression in the `build()` method. The layout decision is made once per build by checking `MediaQuery.of(context).size.width`:

```
final isDesktop = size.width >= 900;

final isTablet = size.width >= 600;

return (isDesktop || isTablet)

  ? Scaffold(

    body: Row(children: [

      _SideNav(...), // Persistent sidebar, always visible

      Expanded(child: Scaffold( // Main content area with its own AppBar

        appBar: AppBar(...),

        body: bodyContent,

        floatingActionButton: FAB,

      )),
```

```

    ),
  )

  : Scaffold(

    drawer: _AppDrawer(...), // Swipe-open drawer for phones

    appBar: AppBar(...),

    body: bodyContent,

    floatingActionButton: FAB,

  );

```

Screen Width	Layout	Navigation
< 600px (phone)	Single column, full width	Hamburger icon opens Drawer from left
600–900px (tablet)	Side navigation + content	Persistent _SideNav widget in a Row
> 900px (desktop/web)	Wide side navigation + content	Full-label persistent sidebar

The **_SideNav** and **_AppDrawer** contain the same navigation items: Home, Book Studio, My Bookings, Profile, Language toggle, Theme toggle, Support, and Sign Out. The sidebar compresses to icon-only mode on tablet width and expands to icons-plus-labels on desktop.

5.5.2 Splash / Welcome Screen UI

The **WelcomeScreen** is the entry point for unauthenticated users. It displays:

- A full-width gradient header panel with the "GENZ Studios" logo and tagline
- A **TabBar** with two tabs: **Sign In** and **Register**
- Sign In tab: email field, password field (with show/hide toggle), "Forgot password" link, and Sign In button
- Register tab: first name, last name, email, password, confirm password fields, and Register button
- Error messages displayed as styled inline banners above the submit button

5.5.3 Home Screen — Welcome Banner

The first visible element on the Home Screen after login is a **full-width gradient welcome banner** — a `Container` with a `BoxDecoration` applying a `LinearGradient` from `AppColors.gradientStart` to `AppColors.gradientEnd`, with a `borderRadius` of `circular(20)`.

Inside the banner:

- **"Welcome back"** in a semi-transparent white 13pt label
- **Client's full name** in bold white 20pt text (`FontWeight.w800`), retrieved from `AWSStorageService.currentUser['name']`
- **"Instant Book" pill button**: a rounded container (`borderRadius: circular(20)`) with `Colors.white.withOpacity(0.2)` background and a `0.3`-opacity white border, containing a `+` icon and "Instant Book" label. Tapping this sets `_selectedIndex = 1`, navigating directly to the booking form.
- **A large `Icons.camera_alt_rounded` icon** at 60px in `Colors.white38` as a decorative visual metaphor for the studio photography theme, positioned at the right end of the banner's `Row`.

5.5.4 Studio Grid — Responsive Display

Below the banner, a section header ("Our Studios" + tagline subtitle) introduces the studio grid. The grid is built with a three-way conditional:

```
size.width > 900

? GridView.count(crossAxisCount: 3, childAspectRatio: 0.82, ...)

: size.width > 600

? GridView.count(crossAxisCount: 2, childAspectRatio: 0.85, ...)

: Column(children: _studios.map(_studioCard).toList())
```

Both `GridView.count` variants use `shrinkWrap: true` and `NeverScrollableScrollPhysics()` because they are embedded inside a parent `SingleChildScrollView` — the outer scroll handles the entire page's vertical scroll, while the grid itself does not scroll independently.

Each studio card (`_studioCard`) is an elevated `Container` with:

- `borderRadius: BorderRadius.circular(20)` — consistent with the app-wide card style
- Theme-aware background color: `AppColors.darkCard` in dark mode, `AppColors.lightSurface` in light mode
- Theme-aware border: `AppColors.darkBorder` / `AppColors.lightBorder` at 1px

- A subtle `BoxShadow` with `Colors.black.withValues(alpha: 0.04)` and `blurRadius: 10` — barely visible in light mode, providing elevation without heavy drop shadow
- Studio image loaded from S3 (displayed at the top of the card)
- Studio name, location, and pricing details below the image
- An availability status indicator (green dot for available, grey for unavailable)
- A "Book Now" action button that pre-selects this studio and navigates to the booking form

5.5.5 Services Catalog — Category-Grouped Design

Below the studio grid, if services are available, the **Services Catalog** section appears. Services are fetched from DynamoDB via AppSync and grouped by category before rendering:

```
final Map<String, List<Map<String, dynamic>>> grouped = {};

for (final svc in _loadedServices) {

    if (svc['available'] == false) continue;

    grouped.putIfAbsent(svc['category'], () => []).add(svc);

}
```

Each category is rendered as a standalone card with:

- A **colored category header** with a tinted background (`accent.withValues(alpha: 0.08)`), a category-specific emoji, the category name in the accent color (`FontWeight.w800`), and a "N packages" count badge
- A list of individual services below the header, each showing name, optional description, and price
- An accent-colored circular dot (8×8px) next to each service name
- A pill-shaped **"Request" button** (`borderRadius: circular(20)`) in the solid accent color that opens a `showModalBottomSheet`

The five service categories each have a distinct color and emoji:

Category	Emoji	Accent Color
Photography Packages	■	<code>Color(0xFF6C63FF)</code> — purple
Video Production	■	<code>Color(0xFF3B82F6)</code> — blue
Advertising & Marketing	■	<code>Color(0xFFFF59E0B)</code> — amber
Creative Design	■	<code>Color(0xFFEF4444)</code> — red

Category	Emoji	Accent Color
Social Media Management	■	Color(0xFF22C55E) — green

The **service request bottom sheet** is launched with `showModalBottomSheet(isScrollControlled: true, backgroundColor: Colors.transparent, ...)`. The `isScrollControlled: true` flag is critical — without it, the bottom sheet's height is constrained to 50% of the screen. With it, the sheet can grow to accommodate the full request form including the soft keyboard, because it handles its own scrolling. The `Colors.transparent` background allows the sheet's own custom `Container` decoration (rounded top corners, theme-aware background) to control the visual appearance without the default grey Material background showing through.

5.5.6 Booking Form — Detailed UI Walkthrough

The booking form (`_buildBookingView`) is the most functionally complex screen. It collects all reservation data through a scrollable form:

Form Field Design

All text input fields use a consistent `InputDecoration` with:

- `filled: true` with `fillColor` set to the current theme's surface color
- `OutlineInputBorder` with `borderRadius: circular(12)` and a 1px theme-aware border
- Floating label behavior (label animates up when the field is focused)

Fields include: First Name, Last Name (side by side in a `Row` on wider screens), Phone Number, Studio Selector (a `DropdownButtonFormField` populated from the live studios list), Start Date, End Date (both using styled `TextFormField` widgets with a `calendar_today_outlined` suffix icon that triggers `_selectDate()`), Start Hour slider, End Hour slider, and Session Duration dropdown.

Date Picker with Business Rules

The date picker is launched via `showDatePicker()` with Flutter's standard date selection UI, but with a custom `selectableDayPredicate` that enforces the studio's Friday closure:

```
selectableDayPredicate: (day) => day.weekday != DateTime.friday,
```

Every Friday in the calendar is greyed out and non-tappable. This is a client-side UX enforcement — the server also validates this rule during atomic booking, but the UI prevents the user from reaching an invalid state before submission.

The date picker also enforces:

- `firstDate: DateTime.now()` — past dates cannot be selected
- `lastDate: DateTime.now().add(Duration(days: 365))` — bookings can be made up to one year in advance

Hour Sliders

Start and end hours are selected using `RangeSlider` or individual `Slider` widgets, clamped to `_openHour = 9` (9:00 AM) and `_closeHour = 19` (7:00 PM). The selected hours are displayed as formatted time labels above the slider that update in real time as the user drags. Hours outside the working window are not selectable — the slider cannot be dragged beyond the clamped range.

Real-Time Price Calculation

As any booking parameter changes (studio selection, dates, hours), `_calculateTotalPrice()` is called inside `setState()`, updating a prominently displayed price summary box at the bottom of the form. The calculation uses the studio's `pricePerHour` value already loaded from DynamoDB — no network call is required:

```
final hours = endHour - startHour;

final days = toDate.difference(fromDate).inDays + 1;

final total = studio['pricePerHour'] * hours * days;

setState(() => estimatedTotal = total);
```

The price summary box shows the breakdown (hours × days × rate) alongside the total in Egyptian Pounds (EGP).

Terms Acceptance

A `CheckboxListTile` displays the terms and conditions acceptance requirement. The submit button is disabled (`onPressed: null`) when `termsAccepted == false`, providing visual feedback that all conditions must be met. The checkbox's active color is `AppColors.primary`.

Atomic Booking Submission Flow

The submission process follows a strict sequence to prevent double-bookings:

1. Client-side validation runs first: all required fields filled, dates valid, `endHour > startHour`, `termsAccepted == true`
2. `setState(() => _isSubmitting = true)` — the submit button is replaced by a `CircularProgressIndicator` to prevent duplicate submissions

3. `AWSStorageService.saveBookingAtomic(booking)` — a server-side GraphQL mutation with a conditional expression that checks for conflicting bookings in DynamoDB before writing
4. On success: two notifications sent via `AWSStorageService.sendNotification()` — one to the employee inbox and one to the client's own notification feed
5. `_clearForm()` resets all form fields and returns to the home tab (`_selectedIndex = 0`)
6. On failure: `_isSubmitting = false` and a descriptive error `SnackBar` appears

Failure reasons are mapped to specific user messages:

```
if (reason == 'studio_booked') {  
  
    msg = loc.translate('studio_booked'); // "Studio already booked"  
  
} else if (reason == 'client_time_conflict') {  
  
    msg = loc.translate('client_time_conflict'); // "You have a booking at this time"  
  
} else if (reason == 'invalid_dates') {  
  
    msg = loc.translate('invalid_dates'); // "Please select valid dates"  
  
} else if (reason.startsWith('auth_error')) {  
  
    msg = 'Session expired. Please login again.';  
  
}
```

5.6 AI-Powered Chatbot — Screen Design & Technical Implementation

The AI chatbot is accessible from **any screen** in the application via a persistent **Floating Action Button** (FAB) rendered in the bottom-right corner of the main `Scaffold`. The FAB uses `Icons.smart_toy_rounded` (a stylized robot icon) on a circular container with `AppColors.primary` background. Its persistence across all tabs ensures the AI assistant is always reachable without navigating away from the current view.

5.6.1 Chatbot Screen UI

Tapping the FAB navigates to `ChatbotScreen` using `MaterialPageRoute`. The screen has a custom `AppBar` that differs from the standard text-only title:

```
AppBar(  
  
  title: Row(children: [  
  
    Container(  
  
      width: 34, height: 34,  
  
      decoration: BoxDecoration(  
  
        gradient: LinearGradient(colors: [  
  
          AppColors.gradientStart, AppColors.gradientEnd]),  
  
        borderRadius: BorderRadius.circular(10),  
  
      ),  
  
      child: Icon(Icons.smart_toy_rounded, color: Colors.white, size: 18),  
  
    ),  
  
    SizedBox(width: 10),  
  
    Column(crossAxisAlignment: CrossAxisAlignment.start, children: [  
  
      Text(chatbot_title, style: TextStyle(fontSize: 15, fontWeight: FontWeight.w700)),  
  
      Text(chatbot_subtitle, style: TextStyle(fontSize: 11, color: subText)),  
  
    ]),  
  
  ]),  
  
  actions: [  
  
    IconButton(icon: Icons.delete_sweep_rounded, onPressed: _clearChat),  
  
  ],  
  
)
```

The AppBar title combines a small gradient-filled square icon (10px border radius) with a two-line title: the chatbot's name on the first line and a subtitle ("Powered by AI" or equivalent) on the second. A delete icon in the actions area triggers the clear-chat flow.

5.6.2 Empty State — Welcome Screen

When no messages have been exchanged yet, the chat area displays a **centered welcome state** instead of an empty `ListView`:

- A large **circular gradient avatar** (80×80px, navy-to-blue gradient, circular box shadow with `blurRadius: 20`) containing a white `smart_toy_rounded` icon at 38px — this serves as the chatbot's visual identity
- A welcome heading in bold 18pt
- A subtitle in 13pt subtext color with `height: 1.6` line spacing
- A `Wrap` of **quick-reply suggestion chips**: "Studio prices?", "Available equipment?", "How to book?", "Working hours?" — each a rounded pill with `AppColors.primary` background at 8% opacity and a 20% opacity border. Tapping any chip calls `_sendMessage(q)` directly, bypassing the text field.

5.6.3 Message Bubbles

The `_bubble()` method renders each message as an `Align` widget with `Alignment.centerRight` for user messages and `Alignment.centerLeft` for bot responses. Each bubble's `Container` has:

- `constraints: BoxConstraints(maxWidth: screenWidth * 0.75)` — bubbles cannot exceed 75% of screen width, maintaining readable line lengths
- `margin: EdgeInsets.only(bottom: 10)` — consistent vertical spacing
- `padding: EdgeInsets.symmetric(horizontal: 14, vertical: 11)`

User message bubble: A `LinearGradient` from `AppColors.gradientStart` to `AppColors.gradientEnd` fills the background. White text. `BorderRadius.only(topLeft: 16, topRight: 16, bottomLeft: 16, bottomRight: 4)` — the bottom-right corner is pinched to visually indicate the user as sender (a convention borrowed from messaging apps).

Bot response bubble: Solid fill with `AppColors.darkCard` (dark mode) or `AppColors.lightSurface` (light mode). Default text color. `BorderRadius.only(topLeft: 16, topRight: 16, bottomLeft: 4, bottomRight: 16)` — bottom-left pinched to indicate the AI as sender. A 1px theme-aware border and a subtle `BoxShadow` distinguish it from the page background.

Both bubble types use `fontSize: 14` with `height: 1.5` line spacing for comfortable reading.

5.6.4 Loading Indicator

While the chatbot server is processing a request, a `LinearProgressIndicator` appears between the message list and the input area:

```

if (_isLoading)

  LinearProgressIndicator(

    color: AppColors.primary,

    backgroundColor: AppColors.primary.withOpacity(0.1),

  ),

```

This is a thin animated bar that runs horizontally across the full screen width, using the primary color with a lightly tinted track. It is more visually subtle than a spinner and does not block the message list — the user can scroll up to read previous messages while waiting.

5.6.5 Message Input Area

The input area at the bottom is a `Container` with a `Border(top: BorderSide(color: borderColor))` separator and theme-aware background. It contains a `Row` with:

- An **expanded rounded text field**: `Container` with `borderRadius: circular(24)`, `Border.all(color: borderColor)`, inside which a `TextField` uses `textInputAction: TextInputAction.send` — pressing Enter on a hardware keyboard or the keyboard's send key dispatches the message without requiring the user to tap the button
- A **circular gradient send button** (44×44px): `BoxDecoration` with the same `LinearGradient` as user bubbles, `shape: BoxShape.circle`, and a `BoxShadow` with `AppColors.primary.withOpacity(0.35)` at `blurRadius: 10` and `Offset(0, 4)`. The `Icons.send_rounded` icon is white, 18px.

5.6.6 Backend Communication & Session Management

Each chatbot session is identified by a **persistent session ID** stored in `SharedPreferences`:

```

String? id = prefs.getString('chatbot_session_id');

if (id == null || id.isEmpty) {

  id = 'sess_${DateTime.now().millisecondsSinceEpoch}';

  await prefs.setString('chatbot_session_id', id);

}

```

This session ID is sent with every message to the Flask server, allowing the server to maintain conversation context (memory) across multiple exchanges in the same session. Each user device has a unique, persistent session.

Messages are sent as HTTP POST to `http://3.239.202.67:5000/chat` with a JSON body:

```
body: jsonEncode({
  'message': text.trim(), // the user's text
  'session_id': _sessionId, // device-specific session
}),
```

The server returns `{ "reply": "..."}` . The application **optimistically adds the user's message to the UI before the server responds** (`setState()` before the `await`), so the user sees their message appear immediately without waiting for the round trip.

The chat history is also persisted locally to `SharedPreferences` as a JSON-encoded list. This means the user sees their previous conversation history when they reopen the chatbot screen, even though the server's conversational memory is the authoritative source for context.

5.6.7 Booking Intent Detection & Execution

When the AI server detects a booking request in the user's message, it embeds a structured booking intent JSON object inside its response. The `ChatbotBookingService.extractBookingIntent(reply)` method parses this embedded JSON. If an intent is found:

1. `ChatbotBookingService.stripBookingIntent(reply)` removes the raw JSON from the visible reply text
2. The cleaned reply is displayed to the user as a regular bot message
3. `_handleBookingIntent(intent)` is called, which loads studio data (if not already loaded) and calls `ChatbotBookingService.confirmBooking(intent, studios)`
4. The booking service resolves the studio name to a real studio object, validates working hours, checks for Friday, calculates the price, and calls `AWSStorageService.saveBookingAtomic()`

5. A confirmation message is added to the chat: "■ Booking confirmed! Studio: X	Date: Y	Duration: Z	Price: N EGP	Booking ID: ..."
---	---------	-------------	--------------	------------------

If the booking fails, a localized error message is displayed (in Arabic if the user's last message was in Arabic, in English otherwise). The language detection uses a Unicode range check: `RegExp(r'[\u0600-\u06FF]').hasMatch(lastUserMessage)`.

5.7 Real-Time Support Chat & Ticket System

The support chat system provides a direct, asynchronous human communication channel between clients and studio staff. Unlike the AI chatbot (which operates entirely independently via the EC2 Flask server), this module is built on AWS AppSync GraphQL and the `ChatMessage` DynamoDB model — meaning messages are persisted to the cloud and visible to all authorized employees simultaneously.

5.7.1 Design Philosophy

The support chat was designed with three principles:

- 1. Client simplicity.** A client should be able to send a message in under two taps from anywhere in the application. The "Support" tab is always present in the navigation drawer and sidebar regardless of screen size or current tab.
- 2. Employee control.** Studio staff receive an aggregated inbox showing threads from all clients, sorted by most recent activity. Employees control whether each client can send messages — this allows staff to close a thread after a matter is resolved, preventing the inbox from filling with follow-up messages for completed cases.
- 3. Persistence.** Unlike a live chat (WebRTC, WebSocket direct), this system persists all messages to DynamoDB. A client's message history survives app restarts, device changes, and employee session changes. An employee picking up a thread can read the full context without asking the client to repeat themselves.

5.7.2 Data Model — ChatMessage

Each `ChatMessage` record in DynamoDB contains:

Field	Type	Description
<code>id</code>	String	UUID, auto-generated by Amplify
<code>senderEmail</code>	String	Email of the message author (client or employee)

Field	Type	Description
senderName	String	Display name of the author
clientEmail	String	Thread identifier — always the client's email, regardless of sender
text	String	Message body text
time	String	ISO 8601 timestamp (e.g., 2024-11-15T14:32:00.000Z)

The `clientEmail` field is used as the thread partition key. All messages in a conversation — both from the client and from employees — share the same `clientEmail` value. To load a thread, the application queries all `ChatMessage` records where `clientEmail == activeClientEmail`.

5.7.3 Client-Side Interface

From the client's navigation, the "Support" item opens a `ChatScreen`. On opening, the screen queries all messages where `clientEmail` matches the signed-in client's email, sorted ascending by `time`. Messages are rendered as a scrollable list of styled bubbles, similar in design to the AI chatbot screen — client messages appear on the right with the application's primary gradient, while staff replies appear on the left with a card-colored background.

The client types in an `InputDecoration`-styled `TextField` at the bottom. Pressing send calls:

```
await AWSStorageService.sendMessage(  
  
  text: _controller.text.trim(),  
  
  clientEmail: currentUser['email'],  
  
  senderEmail: currentUser['email'],  
  
  senderName: currentUser['name'],  
  
);
```

This writes a new `ChatMessage` record via `Amplify.API.mutate()`. The message is immediately added to the local list (optimistic insert) before the AWS call completes, so the UI feels instantaneous.

5.7.4 Employee-Side Interface — Support Inbox

In the employee dashboard, the second bottom navigation tab (labeled "Support") shows the **Support Inbox**. This is a list of all unique client threads, where each entry shows:

- The client's name (resolved via `_clientNameCache`)
- The most recent message snippet and timestamp
- A visual indicator if there are unread messages in the thread

The `_clientNameCache` map is built by scanning the client-sent messages: if a message was sent by a client (determined by cross-referencing `senderEmail` against the non-employee list), the sender's name is stored. This avoids an extra DynamoDB query for a user lookup table.

The employee can perform four actions on each thread:

- **View messages:** tap the thread to open the full conversation, rendered in the same bubble layout as the client sees it
- **Send a reply:** type in the bottom input field; the message is stored with the employee's `senderEmail` but uses the client's `clientEmail` as the thread key
- **Toggle chat access:** a toggle switch on each thread controls whether the client can send new messages. The toggle calls `_toggleChat(email, current)`. Disabling chat prevents the client's `sendChatMessage` calls from succeeding (enforced at the AppSync `@auth` level). Enabling chat sends the client an `AppNotification`: "The support team has opened chat for you."
- **Delete thread:** `_deletePermanently(email)` calls a batch mutation to remove all `ChatMessage` records with `clientEmail == email` from DynamoDB

5.7.5 Optimistic UI for Toggle Actions

The chat enable/disable toggle uses an **optimistic update** pattern to ensure the employee sees an instant response without waiting for the AWS round-trip:

```
Future<void> _toggleChat(String email, bool current) async {

  final next = !current;

  setState(() => _chatEnabledCache[email] = next); // update UI immediately

  await AWSStorageService.enableChatForClient(email, enable: next); // sync to AWS

}
```

The `_chatEnabledCache` is a local `Map<String, bool>` that stores the current chat-enabled state for each client email. By calling `setState()` before the `await`, the toggle switch in the UI flips instantly. If the AWS call fails (network error, timeout), the cache retains the optimistic value — a deliberate UX trade-off, since the cost of a failed toggle (employee sees incorrect state) is lower than the cost of a sluggish UI that makes the employee unsure whether their action registered.

5.7.6 Polling Interval

Messages are refreshed every 10 seconds on the employee side via `_messagesPollingTimer = Timer.periodic(const Duration(seconds: 10), _pollMessages)`. This ensures that new client messages appear on the employee screen within 10 seconds of being sent — an acceptable latency for a human support channel where response times are measured in minutes, not seconds.

5.8 Employee Administration Dashboard

The employee interface (`Employees_screen.dart`) is a full management dashboard for studio operations. It has four primary tabs accessible via a bottom `BottomNavigationBar` or a side drawer: **Dashboard** (booking management), **Support** (client messages), **Studios** (studio CRUD), and **Services** (service catalog CRUD).

5.8.1 Dashboard Tab — Booking Management

The booking management view is the employee's primary operational screen. It displays:

Status filter chips: A horizontal row of tappable chips for filtering bookings by status: All, Pending, Approved, Rejected, Done. The active filter chip is highlighted with `AppColors.primary` background; inactive chips use a tinted outline style. Tapping a chip calls `_applyFilter(filter)`, which updates `filteredBookings` from `_activeBookings` (bookings whose end time has not yet passed and whose status is not "Done").

Summary counters: Two summary cards at the top of the dashboard show the count of Pending and Approved bookings, using color-coded backgrounds (amber for Pending, green for Approved).

Booking cards: Each booking is displayed as a card containing:

- Client name, email, and phone
- Studio name
- Session date range and hours
- Duration and total price
- A color-coded status badge
- Three action buttons: **Approve** (green), **Reject** (red), and **Mark as Done** (blue, for completed sessions)
- A **Delete** icon for removing the booking record entirely

Status update flow: Tapping Approve or Reject opens a `showDialog AlertDialog` with a pre-filled message text field. The default text is a localized approval or rejection message. The employee can

customize this message before confirming. On confirmation, `AWSStorageService.updateBookingStatus(id, newStatus)` writes the new status to DynamoDB, and `AWSStorageService.sendNotification()` delivers the message to the client's notification feed.

Archive section: The `_archivedBookings` getter returns bookings whose `fullEndTime` has passed or whose status is "Done". These are shown in a collapsible section below the active bookings list, sorted newest-first. The archive section is toggled by a `_showArchiveSection` boolean.

5.8.2 Notification System — Employee Side

The employee AppBar contains a notifications bell icon with a **numeric badge** showing unread count:

```
Stack(children: [

  IconButton(icon: Icon(Icons.notifications_rounded), ...),

  if (_unreadNotifsCount > 0)

    Positioned(

      top: 8, right: 8,

      child: Container(

        width: 16, height: 16,

        decoration: BoxDecoration(color: AppColors.error, shape: BoxShape.circle),

        child: Text(

          _unreadNotifsCount > 9 ? '9+' : '$_unreadNotifsCount',

          style: TextStyle(color: Colors.white, fontSize: 9, fontWeight: FontWeight.w800),

        ),

      ),

    ),

])
```

Unlike the client's simple dot indicator, the employee's badge displays the actual numeric count — important for staff managing high booking volumes. It caps at "9+" to fit the 16px circle without overflow.

When a new booking notification arrives during polling, `_showNewBookingBanner()` displays the same custom transparent `SnackBar` banner used on the client side (navy border glow, calendar icon, title, body text, "View" button). Tapping "View" dismisses the banner and switches to the bookings tab.

Tapping the bell opens `_showEmployeeNotifsSheet()` — a modal bottom sheet listing all received notifications with their read/unread state. Notifications are fetched from DynamoDB filtered by `AWSStorageService.employeeInboxKey` (a special email alias for the employee inbox shared across all employees).

5.8.3 Wide-Screen Layout

Like the client screen, the employee dashboard has a responsive wide-screen layout:

```
LayoutBuilder(builder: (context, constraints) {  
  
  final isWide = constraints.maxWidth >= 800;  
  
  if (isWide) {  
  
    return Row(children: [  
  
      _buildSideNav(isDark), // Persistent sidebar  
  
      Expanded(child: _buildCurrentTab()),  
  
    ]);  
  
  } else {  
  
    return _buildCurrentTab(); // Mobile: tab content fills full width  
  
  }  
  
})
```

On screens wider than 800px, a persistent side navigation panel replaces the `Drawer` and `BottomNavigationBar`, providing the same one-click access to all four tabs without the interaction cost of opening a drawer.

5.9 Notifications System

The notifications system is the primary mechanism through which the application keeps users informed about the status of their bookings, support interactions, and account activity. It is implemented as a persistent, cloud-backed in-app notification feed — not a native device push notification system. This architectural decision was deliberate: native push notifications (APNs for iOS, FCM for Android) require additional configuration of platform-specific services, certificate management, and device token registration. The in-app model delivers equivalent functionality using only the existing AppSync and DynamoDB infrastructure already in place.

5.9.1 Architecture — In-App vs Native Push

In a native push notification system, messages are sent from a server to a device-specific token, delivered by Apple's or Google's push infrastructure, and displayed by the OS even when the app is closed. In the Gen Z Studio model, notifications are stored as `AppNotification` records in DynamoDB and retrieved by the application while it is running. This means:

- **Persistence:** A notification sent while the user is offline is not lost — it waits in DynamoDB and appears when the user next opens the app
- **Read state:** Since notifications are database records, the `read` field can be updated, enabling unread/read state management without any OS-level support
- **No platform credentials:** No APNs certificate or FCM server key is required, significantly simplifying the deployment
- **Limitation:** Notifications are not visible when the app is closed. For a studio booking application where sessions are not time-critical at the second level, this trade-off is acceptable

5.9.2 Notification Data Model

Each notification record in DynamoDB contains: `id` (UUID), `clientEmail` (recipient identifier — the email of the user who should see this notification), `title` (short headline, e.g., "Booking Approved"), `body` (full message, e.g., the employee's custom approval message), `type` (category tag: "Approved", "Rejected", "Submitted", "chat_opened", "employee_new_booking"), `time` (ISO 8601 UTC timestamp), and `read` (boolean stored as the string "true" or "false" — the string type was used for DynamoDB compatibility with early Amplify model generation).

The `clientEmail` field is used both as the recipient identifier for client notifications and as a routing identifier for employee inbox entries (using the special value `"employees"` for notifications directed at all employees simultaneously).

5.9.3 Delivery Events

Event	Sender	Recipient	Message
New booking submitted	Client	Employee inbox	"New Booking Request"
New booking submitted	System	Client	"Booking Submitted — awaiting review"
Booking approved	Employee	Client	Custom message from employee
Booking rejected	Employee	Client	Custom message from employee
Chat opened	Employee	Client	"Support team has opened chat for you"

5.9.4 In-App Banner — Custom SnackBar Design

The notification banner is implemented as a `SnackBar` with `backgroundColor: Colors.transparent` and `elevation: 0`, allowing a fully custom-styled `Container` inside:

```
SnackBar(  
  
  duration: Duration(seconds: 5),  
  
  backgroundColor: Colors.transparent,  
  
  elevation: 0,  
  
  padding: EdgeInsets.zero,  
  
  content: Container(  
  
    padding: EdgeInsets.symmetric(horizontal: 16, vertical: 14),  
  
    margin: EdgeInsets.symmetric(horizontal: 12, vertical: 8),  
  
    decoration: BoxDecoration(  
  
      color: isDark ? Color(0xFF1E2D45) : Colors.white,  
  
      borderRadius: BorderRadius.circular(16),  

```

```
border: Border.all(color: AppColors.primary.withValues(alpha: 0.4)),

boxShadow: [BoxShadow(

  color: AppColors.primary.withValues(alpha: 0.15),

  blurRadius: 16, offset: Offset(0, 4),

)],

),

child: Row(children: [

  // Icon in rounded square container

  Container(

    width: 44, height: 44,

    decoration: BoxDecoration(

      color: AppColors.primary.withValues(alpha: 0.12),

      borderRadius: BorderRadius.circular(12),

    ),

    child: Icon(Icons.calendar_today_rounded, color: AppColors.primary, size: 22),

  ),

  SizedBox(width: 14),

  // Title + body text

  Expanded(child: Column(children: [

    Text(title, style: TextStyle(fontWeight: FontWeight.w700, fontSize: 14)),

    Text(body, style: TextStyle(fontSize: 12), maxLines: 2, overflow:
TextOverflow.ellipsis),

  ])),

  // "View" button
```

```

    Container(

      padding: EdgeInsets.symmetric(horizontal: 10, vertical: 6),

      decoration: BoxDecoration(color: AppColors.primary, borderRadius:
BorderRadius.circular(8)),

      child: Text('View', style: TextStyle(color: Colors.white, fontSize: 12,
fontWeight: FontWeight.w700)),

    ),

  ],

),

)

```

This design achieves: a card-style notification that floats above the content, a navy-blue border glow (using `withValues(alpha: 0.4)` — the updated API replacing deprecated `withOpacity`), an icon in a rounded square container, two lines of text with ellipsis overflow on the body, and a solid action button. The 5-second duration gives users enough time to read and respond.

The transparent `SnackBar` approach is specifically chosen over a simple `AlertDialog` because `SnackBar` appears non-blocking — the user can continue reading the current screen while the banner is visible, whereas a dialog demands immediate attention and interaction before the user can proceed.

5.9.5 Notifications Screen — Full Feed View

Beyond the transient in-app banner, the `NotificationsScreen` provides a scrollable list of all past notifications, sorted newest-first. Each entry in the list is rendered as a card showing:

- A type-specific icon and color (green check for Approved, red X for Rejected, blue clock for Pending, bell for general)
- The notification title in bold
- The body text in secondary color
- The formatted relative timestamp ("2 hours ago", "Yesterday", or a full date for older entries)
- A subtle unread indicator (a small blue dot on the left edge of the card) for unread notifications

Tapping a notification marks it as read: `AWSStorageService.markNotificationRead(id)` writes `read: "true"` to DynamoDB, and the local `appNotifications` list is updated via `setState()`. The unread dot disappears immediately. Tapping "Mark all as read" iterates the list and fires a mutation for each unread item.

5.9.6 AppBar Notification Badge

The client's AppBar notification icon uses a small red dot (8×8px circle) rather than a numeric badge, as clients receive fewer notifications than employees:

```
if (appNotifications.isNotEmpty)

  Positioned(

    top: -4, right: -4,

    child: Container(

      width: 8, height: 8,

      decoration: BoxDecoration(color: AppColors.error, shape: BoxShape.circle),

    ),

  ),
```

The dot appears whenever any notification exists in `appNotifications`, regardless of read state. It disappears only after all notifications are cleared.

5.9.7 Polling Architecture for Notifications

Client notifications are polled every 20 seconds via `_notifsPollingTimer`. Employee notifications — which include both general business alerts and the client booking request inbox — are polled every 15 seconds, since timely awareness of new bookings is operationally more critical for staff. The polling calls `AWSStorageService.loadNotifications(email)`, which executes a filtered DynamoDB query via AppSync returning only records where `clientEmail == currentUserEmail`. This scoping ensures that each user receives only their own notifications, reinforced by the AppSync `@auth` rule on the `AppNotification` model.

5.10 Analytics & Reporting

The analytics module is accessible exclusively to employees through the `ReportsScreen`. It provides actionable business intelligence data to help studio managers monitor performance, identify trends, and generate financial summaries for management review. This module transforms raw booking data stored in DynamoDB into meaningful visual representations without requiring any external analytics platform.

5.10.1 Purpose and Business Value

For a studio management company, operational data has direct financial implications. A manager who can see that a particular studio generates 40% of weekly revenue — and that it is mostly booked on Tuesday and Wednesday — can make informed decisions about pricing, staffing levels, and marketing focus. The **ReportsScreen** brings these insights directly to a mobile device without requiring the manager to export data to Excel, run SQL queries, or log into a separate business intelligence tool.

5.10.2 Metrics Displayed

The reports screen presents four primary metrics, each visualized with a dedicated chart or data table:

1. Daily Booking Volume (BarChart)

A **BarChart** widget from the **fl_chart** package displays the number of booking submissions per day over the selected time range. Each bar represents one day, with the height proportional to the booking count. The x-axis shows abbreviated day labels (Mon, Tue, etc.) and the y-axis is auto-scaled to the peak value. The bars are colored with the application's navy primary color, with tooltip popups showing exact counts on tap.

2. Revenue Per Studio (BarChart — Grouped)

A second bar chart compares total revenue (sum of **price** fields for Approved and Done bookings) across all studios. This allows management to immediately identify which studios are highest-performing and which are under-utilized. Only confirmed revenue (Approved + Done status bookings) is counted — Pending and Rejected bookings are excluded to prevent inflating projections with unconverted leads.

3. Booking Status Breakdown (PieChart)

A **PieChart** widget divides the total booking count by status: Pending (amber), Approved (green), Rejected (red), Done (navy). Each segment is labeled with its count and percentage. This chart provides a quick health indicator — a high Pending percentage might suggest that employees are not processing bookings quickly enough, while a high Rejected percentage might indicate mismatch between client expectations and studio availability.

4. Top Clients by Spend (Ranked List)

A sorted list of clients, ranked by total confirmed spend, showing client name, email, and cumulative booking value in EGP. This is rendered as a standard **ListView** rather than a chart, as the ranking relationships are more clearly communicated in text than in a visualization when the number of clients is large.

5.10.3 Time Range Filtering

The `_dashboardTab` variable controls three time range views, selected via a `TabBar` at the top of the reports screen:

Tab	Range	Scope
Today	Current calendar date	Bookings where <code>date == today</code>
This Week	Monday to Sunday of current week	Bookings where <code>date</code> falls within the current week
This Month	1st to last day of current month	Bookings where <code>date</code> falls within the current month

Switching tabs triggers a new DynamoDB query with updated date filter parameters. The charts re-render with the new dataset automatically via `setState()`.

5.10.4 fl_chart Integration

The `fl_chart` package provides a Flutter-native charting library with no native dependencies. Unlike web-based chart libraries (Chart.js, D3) that require a `WebView`, `fl_chart` renders charts directly on Flutter's GPU canvas using the same rendering pipeline as all other widgets. This means charts are:

- **Animated:** bars grow from zero on load; pie segments expand into place
- **Interactive:** tooltip callbacks on tap show exact values
- **Theme-aware:** chart colors are set from `AppColors` constants, so they adapt automatically to dark/light mode
- **Performant:** no JavaScript bridge, no `WebView` overhead

5.10.5 PDF Export

The "Export Report" button generates a formatted PDF document from the current screen's data using the `pdf` Flutter package (the `pw` namespace, distinct from Flutter's `pdf` renderer):

```
final pdf = pw.Document();

pdf.addPage(pw.Page(

  pageFormat: PdfPageFormat.a4,

  build: (context) => pw.Column(
```

```
crossAxisAlignment: pw.CrossAxisAlignment.start,

children: [

  pw.Text('Gen Z Studio – Business Report',

    style: pw.TextStyle(fontSize: 22, fontWeight: pw.FontWeight.bold)),

  pw.SizedBox(height: 8),

  pw.Text('Generated: ${DateFormat('dd MMM yyyy').format(DateTime.now())}',

    style: pw.TextStyle(fontSize: 11, color: PdfColors.grey600)),

  pw.SizedBox(height: 20),

  pw.Text('Summary', style: pw.TextStyle(fontSize: 14, fontWeight:
pw.FontWeight.bold)),

  pw.Divider(),

  pw.TableHelper.fromTextArray(

    data: [

      ['Metric', 'Value'],

      ['Total Bookings', '$totalCount'],

      ['Confirmed Revenue (EGP)', '$confirmedRevenue'],

      ['Pending Bookings', '$pendingCount'],

      // ... additional rows

    ],

  ),

],

),

),

));

final bytes = await pdf.save();
```

The generated PDF bytes are saved to the device's documents directory using platform file APIs. Once saved, the document is shared via the system share sheet (`Share.shareXFiles()`), allowing the manager to send it via email, WhatsApp, or any installed sharing target. This enables managers to distribute formal financial reports to ownership or accounting without requiring access to a web portal or desktop computer.

5.10.6 Employee Access Control

The `ReportsScreen` is only navigable from the employee interface. The navigation item is not present in the client's navigation structure. Additionally, the AppSync queries executed by `ReportsScreen` (which read bookings across all clients) are only permitted for users in the `Employees` Cognito group — the `@auth` rules on `BookingRequest` allow Employees to list all records, while Clients can only retrieve their own. A client cannot access reporting data even if they construct a GraphQL query manually.

5.11 Internationalization & Responsive Design

5.11.1 Bilingual Support (English & Arabic)

The application fully supports **English** and **Arabic**. All user-facing strings are managed by the `AppLocalizations` service class, which acts as a key-to-translation map. The `AppLocalizations.delegate` is registered in `MaterialApp.localizationsDelegates`, enabling Flutter's localization infrastructure.

Language switching is live — no app restart required:

```
// In SettingsService:

static final locale = ValueNotifier<Locale>(const Locale('en'));

// On language toggle:

SettingsService.locale.value = Locale('ar'); // rebuilds all ValueListenableBuilder
listeners
```

Every screen that displays user-facing text calls `AppLocalizations.of(context).translate('key')` to retrieve the current-locale string.

Arabic-specific handling:

- The entire layout is automatically mirrored right-to-left via Flutter's built-in RTL support (activated when `Locale('ar')` is set and `GlobalWidgetsLocalizations.delegate` is registered)
- Service names and other localizable content fields have parallel `name` (English) and `nameAr` (Arabic) fields in DynamoDB. The active field is selected at render time: `isAr && nameAr.isNotEmpty ? nameAr : name`
- The booking error messages, chatbot responses, and notification texts are all translated in the respective service classes

5.11.2 Responsive Breakpoints

Breakpoint	Range	Layout Applied
Phone	width < 600px	Single-column, full-width layout; Drawer navigation
Tablet	600px ≤ width < 900px	Persistent sidebar (compact); 2-column studio grid
Desktop	width ≥ 900px	Full sidebar with labels; 3-column studio grid

These breakpoints are checked at every `build()` invocation using `MediaQuery.of(context).size.width`. There is no static configuration — the layout adapts dynamically if the user resizes a web browser window or rotates a tablet.

5.11.3 System Font Scale Clamping

To prevent UI overflow and layout breaks when a user has enabled large system fonts in their device accessibility settings, the application clamps the text scale factor globally:

```
textScaler: TextScaler.linear(mq.textScaler.scale(1).clamp(0.85, 1.2))
```

This allows up to 20% larger text (for accessibility) and prevents text from being scaled below 85% (maintaining readability), while protecting the layout from extreme scaling values that could break card widths or overflow containers.

5.12 Database Schema & Data Models

The data models form the structural backbone of the entire application. They define what data exists, how it is organized, who can access it, and how the Flutter application interacts with it. Gen Z Studio's data layer uses a **GraphQL Schema Definition Language (SDL)** file maintained at `amplify/backend/api/genz/schema.graphql`. The Amplify CLI reads this schema and performs two transformations: it deploys the corresponding DynamoDB tables and AppSync resolvers on AWS, and it generates strongly-typed Dart model classes in `lib/models/`. This dual transformation means the Dart code and the DynamoDB structure are always in sync — any change to the schema is reflected in both the cloud infrastructure and the application code after running `amplify push`.

5.12.1 GraphQL Schema Definition

The schema file defines each model using the `@model` directive, which instructs Amplify to create a DynamoDB table for that type, along with AppSync resolvers for the standard CRUD operations (create, get, list, update, delete) and subscriptions (onCreate, onUpdate, onDelete). The `@auth` directive attaches access control rules to each model.

A simplified excerpt of the schema showing the most critical models:

```
type BookingRequest @model

  @auth(rules: [

    { allow: owner, ownerField: "clientEmail", identityClaim: "email",

      operations: [create, read] },

    { allow: groups, groups: ["Employees"],

      operations: [read, update, delete] }

  ]) {

  id: String! @primaryKey

  clientEmail: String!

  clientName: String!

  clientPhone: String

  studio: String!

  date: String!
```

```
    hours: Int

    price: Float

    equipment: String

    status: String

    fullStartDateTime: String

    fullEndDateTime: String

}

type Studio @model

    @auth(rules: [

        { allow: public, operations: [read] },

        { allow: groups, groups: ["Employees"],

            operations: [create, update, delete, read] }

    ]) {

    id: String! @primaryKey

    name: String!

    description: String

    location: String

    pricePerHour: Float

    pricePerDay: Float

    imageUrl: String

    available: Boolean

}
```

```
type AppNotification @model

@auth(rules: [

  { allow: owner, ownerField: "clientEmail", identityClaim: "email",

    operations: [read, update] },

  { allow: groups, groups: ["Employees"],

    operations: [create, read, delete] }

]) {

  id: String! @primaryKey

  clientEmail: String!

  title: String!

  body: String

  type: String

  time: String

  read: String

}
```

5.12.2 Model Field Descriptions

BookingRequest — The central transactional entity of the application. Every studio reservation creates one record. The `status` field drives the booking lifecycle: it progresses through `"Pending"` → `"Approved"` or `"Rejected"` → `"Done"`. The `fullStartDateTime` and `fullEndDateTime` fields are computed ISO 8601 datetime strings that combine the `date`, `startHour`, and `endHour` into machine-comparable timestamps. These computed fields are critical for the atomic booking conflict detection logic — AppSync resolvers use them in DynamoDB condition expressions to check whether the requested time range overlaps with any existing confirmed booking for the same studio.

Studio — The studio listing entity. Each studio is created and maintained by employees. The `imageUrl` field stores the S3 key (not the full URL) of the studio's primary photograph; the actual URL

is generated at display time via `Amplify.Storage.getUrl(key: imageUrl)`. The `available` boolean is a soft toggle that allows employees to temporarily hide a studio from the booking interface without deleting its record.

AppNotification — In-app alerts for users. The `clientEmail` field doubles as the routing key: for client notifications, it contains the client's actual email; for employee inbox notifications, it contains the special value `"employees"`, allowing the employee dashboard to query for all entries with this key to build the employee notification inbox.

ChatMessage — Support thread messages. The `clientEmail` field is used as the thread identifier regardless of who sent the message (client or employee). This design allows a single DynamoDB query filtered on `clientEmail` to retrieve the complete conversation history from both sides.

ServiceItem — The services catalog entries. The parallel `name` / `nameAr` fields support Arabic localization: the Flutter code selects the appropriate field at render time based on the active locale. The `category` field groups services into the five standard categories (Photography, Video, Advertising, Design, Social Media) used for the grouped services catalog view.

5.12.3 Auto-Generated Dart Models

After running `amplify push`, the Amplify CLI executes `amplify codegen models`, which reads the GraphQL schema and generates Dart classes in `lib/models/`. Each class is a proper Dart model with:

- Typed fields matching the GraphQL definition
- A factory constructor `fromJson(Map<String, dynamic> json)` for deserializing DynamoDB records
- A `toJson()` method for serialization
- A `copyWith()` method for immutable updates
- Equality and hash implementations based on the `id` field

For example, `BookingRequest.fromJson(data)` is called automatically by the Amplify SDK when a GraphQL query response is deserialized — the Flutter code receives a typed `BookingRequest` object rather than a raw `Map<String, dynamic>`. This type safety means misnamed field accesses are caught at compile time, not at runtime when a user is on the booking screen.

5.12.4 Access Control — AppSync @auth Rules

The `@auth` rules in the schema are enforced at the **AppSync resolver layer** — before any DynamoDB call is made. This is a server-side enforcement, independent of the Flutter client. Even if an attacker bypassed the Flutter UI entirely and sent a raw GraphQL request, AppSync would reject unauthorized operations:

Model	Client Permissions	Employee Permissions
BookingRequest	Create own; read own (filtered by <code>clientEmail</code> = JWT email)	Read all; update <code>status</code> ; delete
Studio	Read only (all studios, no write)	Full CRUD
AppNotification	Read own; update own (mark read)	Create for any email; read all; delete
ChatMessage	Create own messages; read own thread	Read all threads; create as employee; delete entire threads
ServiceItem	Read only	Full CRUD

The owner-based rule on `BookingRequest` uses `ownerField: "clientEmail"` and `identityClaim: "email"` — AppSync reads the `email` claim from the JWT token and compares it to the `clientEmail` field of the record. If they do not match, the query returns no results (for list operations) or a `401 Unauthorized` error (for direct get/update/delete operations). This means a client cannot even see other clients' bookings by modifying a GraphQL query parameter — the filtering happens on the server.

5.13 Real-Time Synchronization Strategy

5.13.1 Why DataStore Was Disabled

AWS Amplify DataStore is a local-first sync engine that maintains a local SQLite database on the device and synchronizes it with DynamoDB through AppSync. However, DataStore performs an initial bulk sync of all records it has permission to read. In a schema with owner-based `@auth` rules, DataStore's sync queries are rejected for records that belong to other users — this causes persistent sync errors and incomplete data on the device.

Rather than work around DataStore's limitations, the application was built using direct `Amplify.API.query()` and `Amplify.API.mutate()` calls with `Timer.periodic` polling and GraphQL subscriptions where supported. This is reflected in the code comment in `main.dart`:

```
// ■■ DataStore disabled – owner-auth schema rejects syncBookingRequests/

// syncChatMessages/etc. Using Amplify.API.query/mutate directly + polling timers.
```

5.13.2 Synchronization Parameters

Data	Consumer	Method	Interval
Studios	Client	<code>Timer.periodic</code> polling	90 seconds
Studios	Employee	<code>Timer.periodic</code> polling	90 seconds
Bookings	Client	<code>Timer.periodic</code> polling	12 seconds
Bookings	Employee	<code>Timer.periodic</code> polling	8 seconds
Support messages	Employee	<code>Timer.periodic</code> polling	10 seconds
Employee notifications	Employee	<code>Timer.periodic</code> polling	15 seconds
Client notifications	Client	<code>Timer.periodic</code> polling	20 seconds

Studios poll less frequently (90s) because they change rarely — only when an employee adds, edits, or removes a studio listing. Bookings poll frequently (8–12s) because they are the core operational data and latency in status updates directly affects the user experience.

5.13.3 Smart Change Detection

Polling does not unconditionally call `setState()`. Before updating the widget tree, the polling handlers compare incoming data against the current state:

```
final changed = ids.length != newIds.length ||

ids.any((id) => !newIds.contains(id)) ||

loaded.any((s) {

  final old = studios.firstWhere((o) => o['id'] == s['id'], orElse: () => {});

  return old.isEmpty || old['name'] != s['name'] || old['available'] !=
s['available'];
```

```
});

if (changed) setState(() => studios = loaded);
```

This prevents unnecessary widget rebuilds when the poll returns identical data — a significant optimization when polls fire every 8–15 seconds.

5.13.4 Ghost-Record Prevention

When an employee deletes a studio, DynamoDB propagates the deletion asynchronously. During the propagation window (typically < 1 second, but variable), a polling client might fetch the studio list and receive the deleted studio. To prevent it from briefly reappearing on screen, a `_deletedStudioIds` Set tracks recently deleted studio IDs. Any studio returned by a poll whose ID is in this set is filtered out of the rendered list for a 5-second grace period, after which the ID is removed from the set.

5.14 User Interface Design — Visual Language & Design System

The Gen Z Studio application uses a consistent, professional visual design language across all screens. The design system is codified in `lib/theme/app_theme.dart` and the `AppColors` constants class.

5.14.1 Color System

Token	Light Mode	Dark Mode	Usage
<code>AppColors.primary</code>	Navy blue	Navy blue	Buttons, active states, FAB, links
<code>AppColors.gradientStart</code>	Deep blue	Deep blue	Gradient fills (banner, bubbles, send button)
<code>AppColors.gradientEnd</code>	Dark navy	Dark navy	Gradient fills
<code>AppColors.success</code>	Green <code>#22C55E</code>	Green	Booking confirmed, approved status, enable toggle
<code>AppColors.error</code>	Red <code>#EF4444</code>	Red	Error messages, reject button, delete, unread badge

Token	Light Mode	Dark Mode	Usage
<code>AppColors.warning</code>	Amber <code>#F59E0B</code>	Amber	Pending status, caution states
<code>AppColors.lightBg</code>	Off-white	—	Screen background (light)
<code>AppColors.darkBg</code>	—	Dark <code>#0F172A</code> approx	Screen background (dark)
<code>AppColors.lightSurface</code> / <code>darkCard</code>	White / Dark card		Card backgrounds, input fields
<code>AppColors.lightBorder</code> / <code>darkBorder</code>	<code>#E2E8F0</code> / <code>#1E293B</code>		Card and input field borders
<code>AppColors.lightSubText</code> / <code>darkSubText</code>	Medium grey	Light grey	Secondary text, placeholders

All `color` references in the widget code use `isDark ? AppColors.darkX : AppColors.lightX` — no screen hardcodes a specific hex value without going through this conditional. This ensures that toggling the theme correctly updates every visual element.

5.14.2 Typography

All headings use `FontWeight.w800` (extra bold) with `letterSpacing: -0.5`. This tight, heavy typographic style is consistent with modern app design conventions and gives the interface a professional, brand-consistent appearance. Body text uses `FontWeight.w400` (normal) with `TA_JUSTIFY` alignment for longer descriptions.

Font sizes follow a clear hierarchy:

- Section headings: 20px
- Card titles / tab labels: 15–16px
- Body / description: 13px
- Metadata / captions / badges: 11–12px

5.14.3 Shape Language

All cards, input fields, bottom sheets, and dialog boxes use `BorderRadius.circular(20)`. Buttons and chips use `BorderRadius.circular(20)` (pills) or `BorderRadius.circular(12)` (buttons). This consistent rounded-corner language gives the interface a soft, approachable character.

Elevation is achieved through `BoxShadow` with very low-opacity black (`alpha: 0.04`) rather than Material's default hard-edge elevation model. This produces a cleaner, more modern-looking depth effect that works equally well in light and dark themes.

5.14.4 Theme Toggle

Dark/light theme preference is persisted to `SharedPreferences` and loaded at startup by `SettingsService.loadSettings()`. The toggle is available from the profile screen for both clients and employees. `SettingsService.themeMode` is a `ValueNotifier<ThemeMode>` that triggers only the `ValueListenableBuilder` wrapping `MaterialApp` — the entire app repaints in the new theme without any screen needing to explicitly handle the transition.

5.15 Summary and Conclusions

The Gen Z Studio Mobile Application represents the culmination of careful technology selection, architectural planning, and user-centered design. This chapter has presented a comprehensive analysis of every layer of the application — from the theoretical underpinnings of the Flutter framework to the concrete DynamoDB schema definitions, from the responsive CSS-like breakpoints in the layout system to the precise timing intervals of the polling timers.

5.15.1 Achieved Objectives

The application successfully delivers on all six primary project objectives:

1. Cross-Platform Reach from a Single Codebase.

Using Flutter and Dart, a single application serves Android phones, iOS devices, and web browsers. There is no separate Android app, iOS app, or web portal — one codebase, one team, one maintenance burden. The responsive layout system adapts to phone (< 600px), tablet (600–900px), and desktop (> 900px) screen sizes automatically, providing an appropriate experience on each form factor.

2. Secure, Role-Based Authentication.

Amazon Cognito handles the full authentication lifecycle — registration, email verification, JWT issuance, automatic token refresh, and secure device-local storage. Two Cognito User Pool Groups enforce role separation: Clients have narrowly scoped permissions (own bookings, own notifications), while Employees have administrative access. Security is enforced at three independent layers: the Flutter routing logic, the AppSync `@auth` resolver rules, and the DynamoDB condition expressions — a defense-in-depth architecture where the compromise of any single layer does not expose data.

3. Real-Time Operational Awareness.

Studio employees receive new booking notifications within 8 seconds of submission (via the bookings polling timer), see new client support messages within 10 seconds, and receive administrative notifications within 15 seconds. Clients see booking status updates within 12 seconds and notification updates within 20 seconds. These latencies are achieved through a carefully designed polling architecture using `Timer.periodic` — a practical solution that avoids the complexity of managing persistent WebSocket connections while delivering near-real-time behavior for a studio management workflow.

4. Conflict-Free Booking.

The `saveBookingAtomic()` function prevents double-bookings through server-side conflict detection using DynamoDB conditional writes. No two bookings can overlap for the same studio, regardless of how many clients submit simultaneously. The client-side form additionally enforces business rules (no Fridays, 9 AM–7 PM operating hours, date range validation, all required fields) before submission, providing immediate feedback without a round-trip to the server.

5. AI-Powered Booking Assistance.

The AI chatbot, hosted on AWS EC2 as a Python Flask service, provides natural language booking assistance in both English and Arabic. The `ChatbotBookingService.extractBookingIntent()` parser detects embedded JSON booking intents in chatbot responses and pre-populates the booking form automatically, bridging the gap between conversational AI and structured data entry.

6. Professional Employee Administration.

The employee dashboard provides full lifecycle management of the booking pipeline (Pending → Approved / Rejected → Done → Archived), a client support inbox with per-thread message management, studio and service catalog CRUD, and a business analytics module with PDF export. The wide-screen layout ($\geq 800\text{px}$) expands the dashboard into a two-column management console, maximizing screen real estate on tablets and desktop browsers.

5.15.2 Technology Selection Retrospective

The core technology decisions, evaluated in hindsight:

Flutter over React Native: The decision proved correct. The official AWS Amplify Flutter SDK provided type-safe, maintained integrations for Cognito, AppSync, and S3 with minimal boilerplate. The AOT compilation delivered smooth 60fps animations across Android and iOS without performance tuning. Hot Reload reduced UI iteration time significantly. The single codebase delivered Android, iOS, and web targets simultaneously.

Direct API calls over DataStore: The decision to disable DataStore was initially a constraint (owner-auth incompatibility), but proved architecturally cleaner. Direct `Amplify.API.query()` and `Amplify.API.mutate()` calls are explicit, predictable, and easier to reason about than DataStore's background sync engine. The polling-based refresh model is transparent — the exact data freshness is known (8–90 seconds depending on data type), whereas DataStore's sync timing is opaque.

DynamoDB via AppSync over SQL: The schema-first GraphQL approach allowed rapid iteration — adding a new field to `BookingRequest` required a schema change and `amplify push`, which automatically updated both the DynamoDB table and the generated Dart models. The declarative `@auth` rules eliminated hundreds of lines of custom authorization middleware.

5.15.3 Technical Challenges and Solutions

Challenge	Root Cause	Solution Applied
DataStore sync errors	Owner-auth <code>@auth</code> rules reject bulk sync queries	Disabled DataStore; switched to direct <code>Amplify.API</code> calls
Double-booking race condition	Two clients submitting simultaneously for the same slot	Server-side atomic write with DynamoDB conditional expression
Ghost records after studio deletion	DynamoDB propagation delay in polling window	<code>_deletedStudioIds</code> Set filters deleted IDs for 5-second grace period
Text overflow at large system fonts	iOS/Android accessibility font scale can reach 200%+	Global text scale clamped to 0.85–1.20× via <code>TextScaler.linear.clamp()</code>
Arabic layout mirroring	Arabic reads right-to-left, Flutter defaults to LTR	<code>GlobalWidgetsLocalizations.delegate</code> registered; <code>Locale('ar')</code> activates built-in RTL
Chatbot language detection	Response language should match the user's last message language	<code>RegExp(r'[\p{Ar}]{1,}')</code> . <code>hasMatch(lastUserMessage)</code> detects Arabic Unicode range
Chatbot session persistence	Each app restart would lose conversation context	Session ID stored in <code>SharedPreferences</code> as <code>chatbot_session_id</code> ; sent with every message

5.15.4 Platform Deployment Summary

The complete deployment stack for the Gen Z Studio Mobile Application:

Component	Technology	Deployment Target
Cross-platform UI framework	Flutter (Dart) — AOT compiled	Android APK / iOS IPA / Web bundle
Authentication & identity	Amazon Cognito User Pool	AWS managed service (Middle East region)
GraphQL API & real-time	AWS AppSync + WebSocket	AWS managed service
Database	Amazon DynamoDB (NoSQL)	AWS managed service (auto-scaled)
File and image storage	Amazon S3	AWS managed service
AI Chatbot backend	Python Flask	AWS EC2 instance (Ubuntu, <code>t3.micro</code>)
State management	<code>setState</code> + <code>Provider</code> + <code>ValueNotifier</code>	Client-side, in-memory
Internationalization	Custom <code>AppLocalizations</code> (EN / AR)	Client-side, bundle
Analytics visualization	<code>fl_chart</code> Flutter package	Client-side, GPU rendered
PDF report generation	<code>pdf</code> Flutter package (<code>pw</code> namespace)	Client-side, device storage
Local persistence	<code>shared_preferences</code>	Device-local (Android SharedPreferences / iOS UserDefaults)

The Gen Z Studio Mobile Application is production-ready across Android, iOS, and Web. It combines a carefully architected Flutter frontend with AWS cloud services to deliver a complete, secure, and user-friendly studio management platform — a system where clients book with confidence and employees operate with clarity.